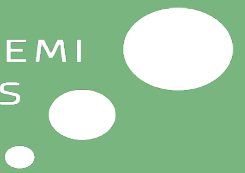
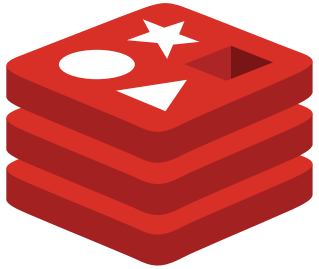


Caching med Redis



Hvad er Redis?

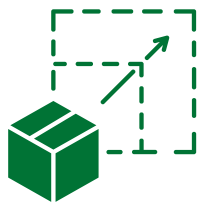


- Open-source in-memory database og cache-løsning.
- Bruger key-value lagring, hvilket gør det hurtigt og effektivt.

Fordele ved Redis som cache:

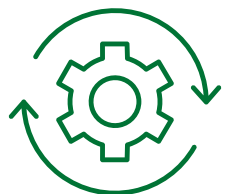
- Hurtigere dataadgang
- reducerer belastningen på databasesystemet.
- Velegnet til brug i distribuerede cloud-native miljøer som Kubernetes.

Hvorfor bruge Redis i K8s?



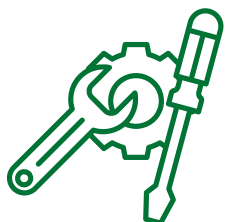
Skalering:

Kubernetes hjælper med at skalere Redis-instanser for at håndtere stigende belastning.



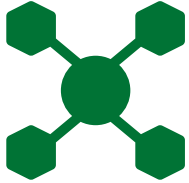
Resiliens:

Kubernetes kan automatisk genstarte **Redis-pods** ved fejl.



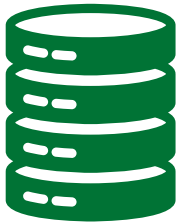
Konfiguration:

Gør det nemmere at administrere Redis-konfiguration gennem Kubernetes konfigurationsfiler.



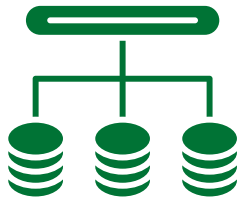
Pods og Services:

Redis kører som en eller flere ***Pods*** og kan forbindes med andre applikationer i klyngen.



Persistent Storage:

Opsætning af ***persistent storage***, så data bevares, selvom en pod genstartes.



Horizontal skalering:

Brug af Redis Cluster til at opdele data og forbedre ydeevnen.



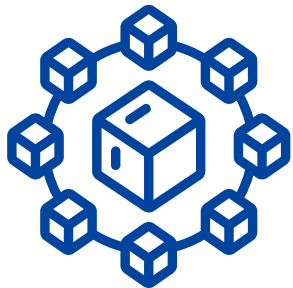
1. Helm Chart:

Brug et Redis ***Helm Chart*** til hurtigt at installere og konfigurere Redis i klyngen.



2. StatefulSets/Secrets:

Brug StatefulSets for at sikre, at Redis-instanser har unikke identiteter og data persistens.



3. Opret Service:

Definér en Kubernetes Service for at tillade andre applikationer at tilgå Redis.



Monitoring:

Brug værktøjer som Prometheus og Grafana til at overvåge Redis-ydeevne.



Horisontal og vertikal skalering:

Justér ressourcer baseret på belastning og brug.



Sikkerhed:

Beskyt Redis ved at begrænse adgang gennem Kubernetes Network Policies.

Redis Helm Chart

```
$ helm install testserver -n caching --create-namespace \
  oci://registry-1.docker.io/bitnamicharts/redis
```

Redis can be accessed on the following DNS names from within your cluster:

testserver-redis-master.caching.svc.cluster.local for read/write operations (port 6379)

testserver-redis-replicas.caching.svc.cluster.local for read-only operations (port 6379)

To get your password run:

```
export REDIS_PASSWORD=$(kubectl get secret --namespace caching testserver-redis \
  -o jsonpath="{.data.redis-password}" | base64 -d)
```

Note: Redis brugernavn er "default"

Kubernetes secrets

```
#####
```

```
# Secrets
```

```
#####
```

```
apiVersion: v1
```

```
kind: Secret
```

```
metadata:
```

```
  name: redis-credentials
```

```
type: Opaque
```

```
data:
```

```
  user: ZGVmYXVsdA==
```

```
  password: TVlQQTIyNzVXMHJEMTIzNA==
```

```
---
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
...
```

```
spec:
```

```
  containers:
```

```
    - name: weatherforecast-pod
```

```
      image: hnrkjnsn/testservice:latest
```

```
      imagePullPolicy: Always
```

```
      env:
```

```
        - name: REDISUSER
```

```
          valueFrom:
```

```
            secretKeyRef:
```

```
              name: redis-credentials
```

```
              key: user
```

```
        - name: REDISPASSWORD
```

```
          valueFrom:
```

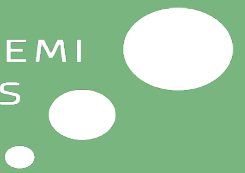
```
            secretKeyRef:
```

```
              name: redis-credentials
```

```
              key: password
```

```
Bash terminal: $ echo -n MYPA$$W0rD1234 | base64
```


Redis med C#



Quickguide C# + Redis

Install NuGet Package

```
$ dotnet add package NRedisStack
```

Program.cs

```
using StackExchange.Redis;
using NRedisStack;

var redisEndpoint = builder.Configuration["RedisConnectionString"] ?? "localhost:6379";
var password = builder.Configuration["REDISPASSWORD"] ?? "";

builder.Services.AddStackExchangeRedisCache(options =>
{
    var configOptions = new ConfigurationOptions
    {
        EndPoints = { redisEndpoint },
        Password = password,
        ConnectTimeout = 5000,
        SyncTimeout = 5000
    };
    options.ConfigurationOptions = configOptions;
    options.InstanceName = "TestServer_";
});
```

WeatherforecastAPI → Minimal API Klasse

Program.cs

```
...  
  
builder.Services.AddSingleton<WeatherforecastAPI>();  
  
...  
app.MapGet("/weatherforecast", async (WeatherforecastAPI api) =>  
    await api.GetWeatherForecastAsync())  
    .WithName("GetWeatherForecast")  
    .WithOpenApi();
```

WeatherforecastAPI.cs

```
public WeatherforecastAPI(IDistributedCache cache)  
{  
    _cache = cache;  
}  
  
public async Task<IResult> GetWeatherForecastAsync()  
{  
    var cacheKey = "weather_forecast";  
    var cachedData = await _cache.GetStringAsync(cacheKey);  
    if (!string.IsNullOrEmpty(cachedData))  
    {  
        return Results.Ok(serializedCachedData);  
    }  
  
    var options = new DistributedCacheEntryOptions  
    {  
        SlidingExpiration = TimeSpan.FromMinutes(1)  
    };  
  
    await _cache.SetStringAsync(cacheKey, newSerializedData, options);
```

Weatherforecast + Instrumentation

Program.cs

```
builder.Services.AddSingleton<Instrumentation>();

// Add OpenTelemetry services
builder.Services.AddOpenTelemetry()
    .WithMetrics(builder =>
    {
        ...
        builder.AddMeter(Instrumentation.MeterName);
        ...
    });
```

Weatherforecast.cs

```
public WeatherforecastAPI(Instrumentation instrumentation)
{
    ArgumentNullException.ThrowIfNull(instrumentation);
    this.activitySource = instrumentation.ActivitySource;
    this.CacheHitCounter = instrumentation.CacheHitCounter;
    this.CacheMissCounter = instrumentation.CacheMissCounter;
}
```

```
public async Task<IResult> GetWeatherForecastAsync()
{
    CacheHitCounter.Add(1);
}
```

```
public class Instrumentation : IDisposable
{
    internal const string ActivitySourceName = "Weatherforecast.Service";
    internal const string MeterName = "Weatherforecast.Service.Metrics";
    private readonly Meter meter;

    public Instrumentation()
    {
        string? version = typeof(Instrumentation).Assembly.GetName().Version?.ToString();
        this.ActivitySource = new ActivitySource(ActivitySourceName, version);
        this.meter = new Meter(MeterName, version);
        this.CacheHitCounter = this.meter.CreateCounter<long>("weatherforecast.cache_hits",
                                                                description: "hits on the cached data");
        this.CacheMissCounter = this.meter.CreateCounter<long>("weatherforecast.cache_misses",
                                                                description: "misses on the cached data");
    }

    public ActivitySource ActivitySource { get; }

    public Counter<long> CacheHitCounter { get; }
    public Counter<long> CacheMissCounter { get; }
}
```

Tilføj Redis server og instrumenteringsklasse til jeres søgemaskine

Eksempel

<https://github.com/HNRK-JNSN/k8s-performance.git>

```
git checkout feature/redis-cache-service
```



Gruppe